

```
Windows PowerShell x Windows PowerShell x Windows PowerShell x mongosh mongodb://127.0.0.1:27017/
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh --port 27017
Current Mongosh Log ID: 6a05a8b64798843994abc113
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.5
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----

The server generated these startup warnings when booting
2026-05-14T16:18:03.002+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-05-14T16:18:03.003+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_i
p <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> rs.initiate()
{
  info2: 'no configuration specified. Using a default configuration for the set',
  me: 'localhost:27017',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778755775, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778755775, i: 1 })
}
rs0 [direct: secondary] test>
```

```
Windows PowerShell x Windows PowerShell x Windows PowerShell x mongosh mongodb://127.0.0.1:27017/
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh --port 27017
Current Mongosh Log ID: 6a05a8b64798843994abc113
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.5
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----

The server generated these startup warnings when booting
2026-05-14T16:18:03.002+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-05-14T16:18:03.003+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_i
p <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> rs.initiate()
{
  info2: 'no configuration specified. Using a default configuration for the set',
  me: 'localhost:27017',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778755775, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778755775, i: 1 })
}
rs0 [direct: secondary] test> rs.add("localhost:27018")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778755837, i: 1 }),
    signature: {

```

```
operationTime: Timestamp({ t: 1778755775, i: 1 })
}
rs0 [direct: secondary] test> rs.add("localhost:27018")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778755837, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778755837, i: 1 })
}
rs0 [direct: primary] test> rs.add("localhost:27019")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778755851, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778755851, i: 1 })
}
rs0 [direct: primary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-05-14T10:51:01.739Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },

```

```

    lastCommittedOpTime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
    lastCommittedWallTime: ISODate('2026-05-14T10:50:53.141Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
    durableOpTime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
    writtenOpTime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
    lastAppliedWallTime: ISODate('2026-05-14T10:50:53.141Z'),
    lastDurableWallTime: ISODate('2026-05-14T10:50:53.141Z'),
    lastWrittenWallTime: ISODate('2026-05-14T10:50:53.141Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1778755826, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-05-14T10:49:35.914Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1778755775, i: 1 }), t: Long('-1') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1778755775, i: 1 }), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1778755775, i: 1 }), t: Long('-1') },
    numVotesNeeded: 1,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    newTermStartDate: ISODate('2026-05-14T10:49:35.978Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-05-14T10:49:36.028Z')
  },
  members: [
    {
      _id: 0,
      name: 'localhost:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 179,
      optime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-05-14T10:50:53.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
      optimeWrittenDate: ISODate('2026-05-14T10:50:53.000Z'),
      lastAppliedWallTime: ISODate('2026-05-14T10:50:53.141Z'),
      lastDurableWallTime: ISODate('2026-05-14T10:50:53.141Z'),
      lastWrittenWallTime: ISODate('2026-05-14T10:50:53.141Z'),
      syncSourceHost: ''
    }
  ]
}
```

```
Windows PowerShell x Windows PowerShell x Windows PowerShell x mongosh mongodb://127.0.0.1 x + v
lastWrittenWallTime: ISODate('2026-05-14T10:50:53.141Z'),
syncSourceHost: '',
syncSourceId: -1,
infoMessage: 'Could not find member to sync from',
electionTime: Timestamp({ t: 1778755775, i: 2 }),
electionDate: ISODate('2026-05-14T10:49:35.000Z'),
configVersion: 5,
configTerm: 1,
self: true,
lastHeartbeatMessage: ''
},
{
  _id: 1,
  name: 'localhost:27018',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 23,
  optime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
  optimeDate: ISODate('2026-05-14T10:50:53.000Z'),
  optimeDurableDate: ISODate('2026-05-14T10:50:53.000Z'),
  optimeWrittenDate: ISODate('2026-05-14T10:50:53.000Z'),
  lastAppliedWallTime: ISODate('2026-05-14T10:50:53.141Z'),
  lastDurableWallTime: ISODate('2026-05-14T10:50:53.141Z'),
  lastWrittenWallTime: ISODate('2026-05-14T10:50:53.141Z'),
  lastHeartbeat: ISODate('2026-05-14T10:51:01.161Z'),
  lastHeartbeatRecv: ISODate('2026-05-14T10:51:01.166Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 5,
  configTerm: 1
},
{
  _id: 2,
  name: 'localhost:27019',
  health: 1,
```

```
Windows PowerShell x Windows PowerShell x Windows PowerShell x mongosh mongodb://127.0.0.1 x + v
configVersion: 5,
configTerm: 1
},
{
  _id: 2,
  name: 'localhost:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 10,
  optime: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1778755853, i: 1 }), t: Long('1') },
  optimeDate: ISODate('2026-05-14T10:50:53.000Z'),
  optimeDurableDate: ISODate('2026-05-14T10:50:53.000Z'),
  optimeWrittenDate: ISODate('2026-05-14T10:50:53.000Z'),
  lastAppliedWallTime: ISODate('2026-05-14T10:50:53.141Z'),
  lastDurableWallTime: ISODate('2026-05-14T10:50:53.141Z'),
  lastWrittenWallTime: ISODate('2026-05-14T10:50:53.141Z'),
  lastHeartbeat: ISODate('2026-05-14T10:51:01.165Z'),
  lastHeartbeatRecv: ISODate('2026-05-14T10:51:01.168Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 5,
  configTerm: 1
}
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1778755853, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1778755853, i: 1 })
}
rs0 [direct: primary] test>
```

```
Windows PowerShell (x86) x mongosh mongodb://127.0.0.1:27030 + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh --port 27030
Current Mongosh Log ID: 6a05aadb7a092bd1b6abc113
Connecting to: mongodb://127.0.0.1:27030/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.5
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-14T16:45:31.594+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-05-14T16:45:31.594+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip
<address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

[direct: mongos] test> sh.addShard("shard1/localhost:27021")
{
  shardAdded: 'shard1',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778757357, i: 18 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778757357, i: 18 })
}
[direct: mongos] test> sh.enableSharding("collegeDB")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778757367, i: 21 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778757367, i: 21 })
}
[direct: mongos] test> sh.shardCollection(
  "collegeDB.students",
  { _id: "hashed" }
)
{
  collectionsSharded: 'collegeDB.students',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778757376, i: 37 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778757376, i: 36 })
}
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId('6a05abd342b47915e1741094'),
  clusterId: ObjectId('6a05abd342b47915e1741093')
}
---
shards
[
  {
    _id: 'shard1',
    host: 'shard1/localhost:27021',
    state: 1,
    topologyTime: Timestamp({ t: 1778757356, i: 33 }),
    replSetConfigVersion: Long('-1')
```

```
Windows PowerShell (x86) x mongosh mongodb://127.0.0.1 x + v
]
---
active mongoses
[ { '0.2.0' : 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[
  {
    ns: 'collegeDB.students',
    shards: [
      {
        shardName: 'shard1',
        numOrphanedDocs: 0,
        numOwnedDocuments: 0,
        ownedSizeBytes: 0,
        orphanedSizeBytes: 0
      }
    ]
  }
]
}
---
databases
[
  {
    database: {
      _id: 'collegeDB',
      primary: 'shard1',
      version: {
        uuid: UUID('8e5c57cb-bb1b-4c15-b693-3c545ea10970'),
        timestamp: Timestamp({ t: 1778757367, i: 9 })
      }
    }
  }
]
```

```
Windows PowerShell (x86) x mongosh mongodb://127.0.0.1 x + v
{
  database: {
    _id: 'collegeDB',
    primary: 'shard1',
    version: {
      uuid: UUID('8e5c57cb-bb1b-4c15-b693-3c545ea10970'),
      timestamp: Timestamp({ t: 1778757367, i: 9 }),
      lastMod: 1
    }
  },
  collections: {
    'collegeDB.students': {
      shardKey: { _id: 'hashed' },
      unique: false,
      balancing: true,
      allowMigrations: true,
      chunkMetadata: [ { shard: 'shard1', nChunks: 1 } ],
      chunks: [
        { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard': 'shard1', 'last modified': Timestamp({ t: 1, i: 0 }) }
      ],
      tags: []
    }
  }
},
{
  database: { _id: 'config', primary: 'config', partitioned: true },
  collections: {}
}
]
[direct: mongos] test> use collegeDB
switched to db collegeDB
[direct: mongos] collegeDB> db.students.insertOne({
  name: "Junaid",
  city: "Pune"
})
{
  acknowledged: true,
  insertedId: ObjectId('6a05af237a092bd1b6abc114')
}
[direct: mongos] collegeDB>
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh --port 27021
Current Mongosh Log ID: 6a05aeb03351a09e16abc113
Connecting to:      mongodb://127.0.0.1:27021/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.2.5
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-14T16:43:48.527+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2026-05-14T16:43:48.528+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_i
p <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

shard1 [direct: primary] test>
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh --port 27020
Current Mongosh Log ID: 6a05ae445a4be2d755abc113
Connecting to:      mongodb://127.0.0.1:27020/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.2.5
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-14T16:42:58.866+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2026-05-14T16:42:58.867+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_i
p <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

configRS [direct: primary] test> rs.initiate()
MongoServerError[AlreadyInitialized]: already initialized
configRS [direct: primary] test>
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai\OneDrive\Desktop> cd "C:\Program Files\MongoDB\Server\8.2\bin"
PS C:\Program Files\MongoDB\Server\8.2\bin> mongosh
Current Mongosh Log ID: 6a059ebdea04600ad4abc113
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.5
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-14T15:15:02.261+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test>

test> use collegeDB
switched to db collegeDB
collegeDB> db.createCollection("students")
{ ok: 1 }
collegeDB> show collections
students
collegeDB> db.students.insertOne({
  |   name: "Josh",
  |   age: 21,
  |   city: "Mumbai"
  | })
{
  acknowledged: true,
  insertedId: ObjectId('6a059f3fea04600ad4abc114')
}
collegeDB> db.students.insertMany([
```

```
]
collegeDB> db.students.insertMany([
  | {
  |   name: "Satish",
  |   age: 22,
  |   city: "Mumbai"
  | },
  | {
  |   name: "Saniya",
  |   age: 20,
  |   city: "Delhi"
  | },
  | {
  |   name: "Aman",
  |   age: 24,
  |   city: "Pune"
  | }
  | ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a059f76ea04600ad4abc115'),
    '1': ObjectId('6a059f76ea04600ad4abc116'),
    '2': ObjectId('6a059f76ea04600ad4abc117')
  }
}
collegeDB> db.students.find().pretty()
[
  {
    _id: ObjectId('6a059f3fea04600ad4abc114'),
    name: 'Josh',
    age: 21,
    city: 'Mumbai'
  },
  {
    _id: ObjectId('6a059f76ea04600ad4abc115'),
    name: 'Satish',
    age: 22,
    city: 'Mumbai'
  },
  {
    _id: ObjectId('6a059f76ea04600ad4abc116'),
    name: 'Saniya',
    age: 20,
    city: 'Delhi'
  },
  {
    _id: ObjectId('6a059f76ea04600ad4abc117'),
    name: 'Aman',
    age: 24,
    city: 'Pune'
  }
]
```

```
Windows PowerShell
},
{
  _id: ObjectId('6a059f76ea0460ad4abc116'),
  name: 'Saniya',
  age: 20,
  city: 'Delhi'
},
{
  _id: ObjectId('6a059f76ea0460ad4abc117'),
  name: 'Aman',
  age: 24,
  city: 'Pune'
}
]
collegeDB> db.students.updateOne(
|   { name: "Ali" },
|   {
|     $set: { age: 25 }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
collegeDB> db.students.find({ name: "Ali" })
collegeDB> db.students.updateMany(
|   { city: "Pune" },
|   {
|     $set: { course: "MSc CS" }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
Windows PowerShell
collegeDB> db.students.deleteOne({
|   name: "Saniya"
| })
{ acknowledged: true, deletedCount: 1 }
collegeDB> db.students.createIndex({
|   name: 1
| })
name_1
collegeDB> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1' }
]
collegeDB> db.students.find({
|   city: "Pune"
| })
[
  {
    _id: ObjectId('6a059f76ea0460ad4abc117'),
    name: 'Aman',
    age: 24,
    city: 'Pune',
    course: 'MSc CS'
  }
]
collegeDB> db.students.find({
|   age: { $gt: 20 }
| })
[
  {
    _id: ObjectId('6a059f3fea0460ad4abc114'),
    name: 'Joshi',
    age: 21,
    city: 'Mumbai'
  },
  {
    _id: ObjectId('6a059f76ea0460ad4abc115'),
    name: 'Satish',
    age: 22,
    city: 'Mumbai'
  }
]
```



```
Windows PowerShell
>_id: ObjectId('6a059f76ea0460ad4abc117'),
name: 'Aman',
age: 24,
city: 'Pune',
course: 'MSc CS'
}
]
collegeDB> db.students.find({
  city: "Pune",
  age: { $gt: 20 }
})
[
  {
    _id: ObjectId('6a059f76ea0460ad4abc117'),
    name: 'Aman',
    age: 24,
    city: 'Pune',
    course: 'MSc CS'
  }
]
collegeDB> db.students.find({
  $or: [
    { city: "Pune" },
    { city: "Delhi" }
  ]
})
[
  {
    _id: ObjectId('6a059f76ea0460ad4abc117'),
    name: 'Aman',
    age: 24,
    city: 'Pune',
    course: 'MSc CS'
  }
]
collegeDB> db.students.find().sort({
  age: 1
})
[
  {
    _id: ObjectId('6a059f3fea0460ad4abc114'),
```

```
Windows PowerShell
]
]
collegeDB> db.students.find().sort({
  age: 1
})
[
  {
    _id: ObjectId('6a059f3fea0460ad4abc114'),
    name: 'Josh',
    age: 21,
    city: 'Mumbai'
  },
  {
    _id: ObjectId('6a059f76ea0460ad4abc115'),
    name: 'Satish',
    age: 22,
    city: 'Mumbai'
  },
  {
    _id: ObjectId('6a059f76ea0460ad4abc117'),
    name: 'Aman',
    age: 24,
    city: 'Pune',
    course: 'MSc CS'
  }
]
collegeDB> db.students.find().sort({
  age: -1
})
[
  {
    _id: ObjectId('6a059f76ea0460ad4abc117'),
    name: 'Aman',
    age: 24,
    city: 'Pune',
    course: 'MSc CS'
  },
  {
    _id: ObjectId('6a059f76ea0460ad4abc115'),
    name: 'Satish',
    age: 22,
```

```
Windows PowerShell
},
{
  _id: ObjectId('6a059f3fea04600ad4abc114'),
  name: 'Josh',
  age: 21,
  city: 'Mumbai'
}
]
collegeDB> var cursor = db.students.find()
collegeDB> while(cursor.hasNext()) {
|   printjson(cursor.next())
| }
{
  _id: ObjectId('6a059f3fea04600ad4abc114'),
  name: 'Josh',
  age: 21,
  city: 'Mumbai'
}
{
  _id: ObjectId('6a059f76ea04600ad4abc115'),
  name: 'Satish',
  age: 22,
  city: 'Mumbai'
}
{
  _id: ObjectId('6a059f76ea04600ad4abc117'),
  name: 'Aman',
  age: 24,
  city: 'Pune',
  course: 'MSc CS'
}
]
collegeDB> db.students.aggregate([
| {
|   $group: {
|     _id: "$city",
|     total: { $sum: 1 }
|   }
| }
| ])
[ { _id: 'Mumbai', total: 2 }, { _id: 'Pune', total: 1 } ]
```

```
Windows PowerShell
| }
| ])
[ { _id: 'Mumbai', total: 2 }, { _id: 'Pune', total: 1 } ]
collegeDB> db.students.aggregate([
| {
|   $group: {
|     _id: null,
|     avgAge: { $avg: "$age" }
|   }
| }
| ])
[ { _id: null, avgAge: 22.333333333333332 } ]
collegeDB> db.createCollection("courses")
{ ok: 1 }
collegeDB> db.courses.insertMany([
| {
|   course_id: 101,
|   course_name: "MSc CS"
| },
| {
|   course_id: 102,
|   course_name: "MBA"
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a05a038ea04600ad4abc118'),
    '1': ObjectId('6a05a038ea04600ad4abc119')
  }
}
collegeDB> db.students.updateMany(
| {
|   {
|     $set: { course_id: 101 }
|   }
| }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
}
```

```
Windows PowerShell
[ ]
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
collegeDB> db.students.aggregate([
  {
    $lookup: {
      from: "courses",
      localField: "course_id",
      foreignField: "course_id",
      as: "course_details"
    }
  }
])
[
  {
    _id: ObjectId('6a059f3fea04600ad4abc114'),
    name: 'Josh',
    age: 21,
    city: 'Mumbai',
    course_id: 101,
    course_details: [
      {
        _id: ObjectId('6a05a038ea04600ad4abc118'),
        course_id: 101,
        course_name: 'MSc CS'
      }
    ]
  },
  {
    _id: ObjectId('6a059f76ea04600ad4abc115'),
    name: 'Satish',
    age: 22,
    city: 'Mumbai',
    course_id: 101,
    course_details: [
      {
        _id: ObjectId('6a05a038ea04600ad4abc118'),
        course_id: 101,
        course_name: 'MSc CS'
      }
    ]
  }
]
}
```

```
Windows PowerShell
{
  _id: ObjectId('6a059f76ea04600ad4abc115'),
  name: 'Satish',
  age: 22,
  city: 'Mumbai',
  course_id: 101,
  course_details: [
    {
      _id: ObjectId('6a05a038ea04600ad4abc118'),
      course_id: 101,
      course_name: 'MSc CS'
    }
  ]
},
{
  _id: ObjectId('6a059f76ea04600ad4abc117'),
  name: 'Aman',
  age: 24,
  city: 'Pune',
  course: 'MSc CS',
  course_id: 101,
  course_details: [
    {
      _id: ObjectId('6a05a038ea04600ad4abc118'),
      course_id: 101,
      course_name: 'MSc CS'
    }
  ]
}
]
}
collegeDB> db.createCollection("employees", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "salary"],
      properties: {
        name: {
          bsonType: "string"
        },
        salary: {
          bsonType: "int",

```

```
Windows PowerShell
]
}
}
collegeDB> db.createCollection("employees", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "salary"],
      properties: {
        name: {
          bsonType: "string"
        },
        salary: {
          bsonType: "int",
          minimum: 1000
        }
      }
    }
  }
})
{ ok: 1 }
collegeDB> db.employees.insertOne({
  name: "Rahul",
  salary: 5000
})
{
  acknowledged: true,
  insertedId: ObjectId('6a05a05eea04600ad4abc11a')
}
collegeDB> db.employees.insertOne({
  name: "Aman",
  salary: 500
})
Uncaught:
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('6a05a067ea04600ad4abc11b'),
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {

```

```
Windows PowerShell (x86)
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh
Current Mongosh Log ID: 6a05b0be9d44ab3a38abc113
Connecting to: mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.5
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-14T16:53:27.741+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-05-14T16:53:27.741+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_i
p <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> use admin
switched to db admin
admin> db.createUser({
  user: "admin",
  pwd: "password123",
  roles: [
    {
      role: "root",
      db: "admin"
    }
  ]
})
{ ok: 1 }
admin>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
admin>
PS C:\Users\junai> mongod --auth --dbpath C:\data\db
{"t":{"$date":"2026-05-14T16:54:43.810+05:30"},"s":"I", "c":"","", "id":8991200, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":2740754
328}}
{"t":{"$date":"2026-05-14T16:54:43.839+05:30"},"s":"I", "c":"CONTROL", "id":97374, "ctx":"thread1","msg":"Automatically disabling TLS 1.0 and TLS 1.1, t
```

```
Windows PowerShell (x86) x Windows PowerShell x mongosh mongodb://127.0.0.1:27017/ x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\junai> mongosh
Current Mongosh Log ID: 6a05b13ae9f34ac1aaabc113
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.5
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-14T16:53:27.741+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-05-14T16:53:27.741+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip
<address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, star
t the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> show dbs
admin 100.00 KiB
config 12.00 KiB
local 192.00 KiB
test> use admin
switched to db admin
admin> db.auth("admin", "password123")
{ ok: 1 }
admin>
```